16 - Webové technologie

Webové technologie na straně klienta, pojem frontend

- Webový klient = **prohlížeč** (tzv. *browser*) - např. *Google Chrome*, *Mozilla Firefox*, *Opera*, *Safari*, *Microsoft Edge*, *Internet Explorer* a jiné.
- **Webové technologie na straně klienta -** spuštěny ve chvíli přijetí webové stránky prohlížečem, k svému běhu vyžadují buď jen prohlížeč, nebo i přídavné programové moduly (tzv. **pluginy**). Hlavním úkolem je prezentovat data uživatelům webu a umožnit interaktivní komunikaci s webovým serverem.
- Základní technologie (HTML, CSS, JS), frameworky a knihovny (Bootstrap, jQuery, React, Angular), starší technologie vyžadující pluginy a problematické z pohledu bezpečnosti (Flash, applety v jazyce Java).
- **Frontend** = prezentační vrstva aplikace, co se zobrazuje běžným uživatelům (klientská část). **Frontend vývojáři** (*frontend developer***)** vytváří kód pro uživatelské vizuální prostředí stránky.

Charakteristika jazyka JavaScript

- **JavaScript** je **multiplatformní, objektově-orientovaný skriptovací jazyk**, který je většinou využíván k programování na straně klienta a **interpretován webovými prohlížeči**. Byť v názvu obsahuje slovo Java, nelze ho ztotožňovat s tímto programovacím jazykem. Oficiální název je ve skutečnosti **ECMAScript** (např. *ECMAScript 6*, *ECMAScript 2018*, *ECMAScript 2019*).
- JS se primárně využívá jako **skriptovací jazyk** na webových stránkách. Umožňuje programovat **interaktivní prvky** GUI (tlačítka, formulářová pole), tvořit **animace a grafické efekty**, validovat **formuláře na straně klienta**, využívat **AJAX**.
- Základní **s**yntax je podobná jazykům C (C++) nebo Java (shodné příkazy, složené závorky, komentáře). Důležité rozdíly:
    - **interpretovaný jazyk** nikoliv kompilovaný (zůstává v podobě zdrojového textu)**,**
    - **dynamicky (či slabě) typový jazyk** – typy proměnných v JS závisí na přiřazené hodnotě a mohou být implicitně přetypovány,
    - **objektový jazyk** – objekty jsou proměnné i funkce, starší verze používaly tzv. **prototypy**, nové verze podporují třídy - **class**,
    - využití anonymních **callback funkcí (**nověji i tzv. **arrow functions)**, které se vkládají jako parametry do jiných funkcí (např. reakce na určitou událost - akci klávesnice či myši).

DOM a BOM

- **DOM** (*Document Object Model*) **objektový model HTML stránky**, který je pojímán jako **stromová struktura objektů-uzlů** (*node-tree*). Nejvýše je **kořenový element** (*root*) – **<html>**. Jednotlivé uzly mohou představovat buď **značky** (*element nodes*), **texty** (*text nodes*) nebo **atributy** (*attribute nodes*). **Rodičovské uzly** (*parentNode*) jsou podřízeny **uzly potomků** (*childNodes*).
- Objekty mají své **vlastnosti** (*properties*), **metody** (*methods*), případně i **události** (*events*). Události mohou být na webové stránce vyvolány interakcemi uživatele (myš, klávesnice), ale také interakcí prostředí (načítání stránky, chybovou událostí). Událost je detekována tzv. "naslouchačem" (*EventListener*), který vyvolává ohlasovou metodu (v JS anonymní callback funkci).
- **BOM** (*Browser Object Model) je* **objektový model prohlížeče**. Vychází ze základního objektu ***window***, který odkazuje na okno prohlížeče. Jiné objekty zase například umožňují přistupovat k atributům obrazovky (***screen***) nebo k historii prohlížených stránek (***history***).
- Jak k DOM, tak i k BOM můžeme přistupovat pomocí JS.

Javascriptové knihovny, charakteristika knihovny jQuery

- **Knihovna** představuje v softwarovém pojetí **souhrn předem napsaných kódů v podobě funkcí i celých objektů**, které je možné využívat při určitých programovacích operacích.

- Programátorům v jazyce JavaScript jsou k dispozici stovky knihoven pro nejrůznější účely. Příkladem mohou být: *Chart.js (vykreslování grafů), Matter.js* (simulace fyzikálních vlastností objektů včetně gravitace), *Three.js* (vytváření animací a simulací v trojrozměrném prostoru), *TensorFlow.js* (podpora umělé inteligence - tzv. strojového učení).

- Připojení knihoven do frontendové aplikace spočívá buď v jejich stažení přímo do některé složky projektu a vložení relativní URL adresy do značky <script>, nebo vložení absolutní URL adresy mířící obvykle na některý z tzv. **CDN serverů** (*Content Delivery Network*) - bývá velmi často využívána k zpřístupnění open-source knihoven.

- **jQuery** je jedna z nejpoužívanějších JS knihoven, která usnadňuje manipulaci s objekty i tvorbu animací na webových stránkách. Pro spolehlivé fungování kódu jQuery je nezbytné, aby byl spuštěn teprve poté, když je načtena („připravena“) celá stránka - k tomu slouží speciální *anonymní funkce*:

  $(document).ready(function(){ /* kód využívající jQuery*/});

- Klíčovou roli hrají v jQuery **selektory** dobře známé z CSS, pomocí kterých lze spolehlivě identifikovat libovolný objekt v rámci webové stránky (DOM). Pro odlišení jQuery selektorů od běžného javascriptového kódu se používá dolarová notace - **$(selektor).** Vyhledání určitého objektu lze dosáhnout také pomocí tzv. **traverzování ve stromu objektů**, které vyplývá ze vzájemných vztahů objektů jako *rodičů*, *potomků* nebo symbolických *sourozenců*.

Webové technologie na straně serveru, pojem backend

- **Webové technologie na straně serveru** mají zajistit reakci na požadavky klientů a připravovat statický i dynamický obsah webu. K nejčastěji používaným **webovým serverům** patří programy *Apache*, *Nginx*, *MS Internet Information Service*.

- Pro generování dynamického obsahu, ale i shromažďování dat se na straně serveru využívají **různé programovací jazyky** (PHP, Python, Java, C#, JavaScript atd.), **frameworky**, **databázové systémy** (MySQL, PostgreSQL, MongoDB atd.) a další softwarové prostředky.

- **Backend =** vrstva operující se samotnými daty softwaru, fyzické infrastruktury nebo hardwaru. **Backend vývojář** (*backend developer*) je typ programátora, který se stará o zásadní logiku aplikace. Práce předpokládá nejen dobrou znalost programovacích jazyků, ale rovněž databázových systémů a různých datových formátů.

Příklady často využívaných technologií na straně serveru

- **PHP** je skriptovací programovací jazyk interpretovaný na straně serveru a určený primárně k programování webových aplikací. PHP prochází neustálým vývojem, jednotlivé verze se od sebe poměrně výrazně liší; v současnosti existuje již PHP verze 8. Kód PHP zapisujeme mezi značky **<?php...?>** do souborů s příponou **.php**. V PHP byly napsány i velké internetové projekty, včetně *Wikipedie* nebo *Facebooku*. V jazyce PHP byly vytvořeny rovněž nejrozšířenější CMS systémy jako *Wordpress*, *Drupal* nebo *Joomla*. Pro vývoj rozsáhlejších aplikací se využívají PHP frameworky, k nimž patří například *Laravel*, *Symphony* nebo český *NetteFramework*.

- **Node.js** je multiplatformní a open-source běhové prostředí (runtime environment) určené pro jazyk JavaScript. Díky jemu je možné tento populární skriptovací jazyk používat i pro vytváření webových aplikací na straně serveru. Na rozdíl od PHP, které podporuje provádění synchronních operací (viz níže), Node.js využívá **asynchronní přístup**, což umožňuje vytvářet i velmi náročné internetové aplikace schopné zpracovávat velké množství požadavků. Díky vestavěnému správci balíčků **NPM** (*Node.js package manager*) mohou vývojáři ve svých aplikacích instalovat a využívat množství externích modulů pro nejrůznější účely.

- K tvorbě velkých podnikových informačních systémů, k nimž jejich uživatelé přistupují prostřednictvím internetu i intranetu, se často využívají technologie podporující objektově-orientované programování v různých jazycích (C++, C#, Java, Kotlin). Příkladem je **platforma .NET** od společnosti Microsoft tvořená programovacími nástroji a knihovnami pro vytváření různých typů aplikací. Pro spuštění .NET aplikací je nutné mít v operačním systému nainstalován .NET Framework (bývá integrován s novějšími OS Windows, v současnosti lze naistalovat i v Linuxu). K vytváření webových aplikací se používá rozšíření (sada speciálních knihoven) **ASP.NET**. Podobný koncept nabízí firma Oracle v podobě technologie **JSP** (*JavaServer Pages*).

Synchronní a asynchronní operace, AJAX

- Zatímco u běžnějších **synchronních operací** (například vyhodnocení matematického výrazu) program vždy čeká na jejich dokončení a teprve poté pokračuje, v případě **asynchronní operace** program vyvolá tzv. callback funkci a hned pokračuje dalšími příkazy. Ve chvíli, kdy je výstup callback funkce připraven, vrátí se k němu i pozornost programu.

- *Pro lepší pochopení lze použít příklad dvou různých typů restaurací. V **"synchronní" restauraci** by číšník zapsal objednávku zákazníka, poté ji předal kuchyni, počkal, až kuchyně jídlo připraví, přinesl jídlo hostu a teprve poté mohl přijmout další objednávku. V **"asynchronní" restauraci** se číšník chová tak, jak jsme zvyklí. Zapíše objednávku jednoho hosta a předá pokyn kuchyni. Nečeká však, až se jídlo připraví, ale řeší další objednávky dalších zákazníků a jakmile je připraveno některé z jídel, přináší je na určený stůl. Z toho je jasné, že asynchronní přístup je efektivnější, neboť jeden číšník dokáže obsloužit ve stejném čase mnohem více hostů a není "blokován" čekáním na přípravu jídla. Na druhou stranu ale nemusí být předem jasné, v jakém pořadí budou hosté obslouženi (méně náročné jídlo může být připraveno dříve, i když bylo objednáno později).*

- Synchronní přístup je typický například pro aplikace vytvořené v PHP - jsou proto náročnější na výkon serveru, který musí požadavky většího počtu uživatelů řešit použitím **multithreadingu** (rozdělení běhu aplikace do více tzv. vláken). JavaScript (a Node.js na straně serveru) je označován za **jednovláknový neblokující asynchronní jazyk**, který k řešení požadavků používá callback funkce.

- **AJAX** (*Asynchronous JavaScript and XML*) je označení pro technologie vývoje interaktivních webových aplikací, které mění obsah stránky bez opětovného načítání celého webového dokumentu. Toho se dosahuje pomocí *JavaScriptu*, který získává libovolná data od webového serveru, dříve nejčastěji ve formátu XML, v současnosti nejčastěji **JSON**.

- Na rozdíl od klasických webových aplikací poskytují stránky s AJAXem uživatelsky příjemnější prostředí. Pokud například uživatel klikne na tlačítko pro udělení hlasu v nějaké anketě, celá stránka se musí znovu načíst ze serveru, třebaže se na ní jen například aktualizují výsledky hlasování a veškerý zbytek obsahu zůstává stejný. Prostřednictvím AJAXu proběhne odeslání hlasu uživatele na pozadí, server zašle jen ty části stránky, které se změnily, a jen tyto části se uživateli na stránce aktualizují a překreslí. S AJAXem má uživatel pocit mnohem větší plynulosti práce, která se (zejména u rychlejšího internetového připojení) blíží běžným desktopovým aplikacím. Z toho vyplývá také možnost snížení zátěže na webové servery a síť obecně.

- Mezi nevýhody patří hlavně principiální změna používání webu: webové stránky se chovají jako plnohodnotné aplikace se složitou vnitřní logikou, nikoli jako posloupnost stránek, mezi kterými se lze navigovat i pomocí tlačítek Zpět a Další. Není také možné snadno předat URL stránky, na níž byly provedeny změny prostřednictvím AJAXu. Problémem AJAXových aplikací může být i **komunikace se serverem v pozadí aplikace**: pokud uživatel neobdrží zřetelný signál (např. animovanou ikonkou), že probíhá odbavení požadavku, může reagovat nevhodně - např. opakovat objednávku vstupenek. Další nevýhodou AJAXu je **nutnost používat moderní grafické prohlížeče**, které podporují potřebné technologie (nemusí například fungovat v některých starších mobilních zařízeních).

Webové frameworky a jejich využití v praxi

- Frameworkem označujeme sadu nástrojů či knihoven, které pomáhají **zjednodušovat a zrychlovat vývojářům tvorbu webových aplikací**. Frameworky obvykle **poskytují základní kostru projektu** a často jsou tvořeny šablonami, které mají specifické vlastnosti a specifická místa, kam a jak psát kód.

- Frameworky můžeme rozdělit do skupin podle toho, kterou oblast vývoje webové aplikace pokrývají nebo v jakém programovacím jazyce vznikly. V praxi se tak setkáváme jak s **frontend i backend frameworky**, frameworky vytvořené v jazyce Python nebo v jazyce PHP apod.

- Příkladem tzv. **front-end frameworku** je *Bootstrap* (původně *Twitter Bootstrap)*. K využití Bootstrapu na stránce je nutné připojit nebo stáhnout potřebné CSS a JS soubory. Bootstrap umožnuje dosáhnout nejen responzivního designu stránky (základem layoutu je dvanáctisloupcový grid), ale i moderního vzhledu s pokročilými prvky – navigační lišty s rozbalovacím menu, otevírací okna, prvky s dynamicky se měnícím obsahem a mnoho dalších.

- Příkladem **frameworku pro tvorbu webových serverových aplikací** v jazyce Python je **Django**. Jeho součástí je celá sada vestavěných modulů pro přístup k databázím, správu session, autentizaci uživatelů, vytváření a validaci formulářů. Obsahuje **šablonovací systém** umožňující rychlejší vytvoření uživatelského rozhraní webové aplikace i **ORM systém** (objektově relační mapování), který řeší objektový přístup k datům. Nabízí rovněž automatické generování **administračního rozhraní** webu.

- Jiným typem jsou tzv. **mikroframeworky**, mezi které patří například **Flask** používaný pro vývoj webových aplikací v jazyce Python. Mikroframework řeší pouze základní operace spojené se spuštěním aplikace a neobsahuje funkcionality běžných frameworků (např. databázovou vrstvu, správu účtů, šablonovací system apod.). Funkce je však možné libovolně přidávat pomocí rozšíření, což poskytuje vývojářům velkou volnost při vytváření vlastních aplikací a minimalistické řešení se může pozitivně projevit na výkonu.